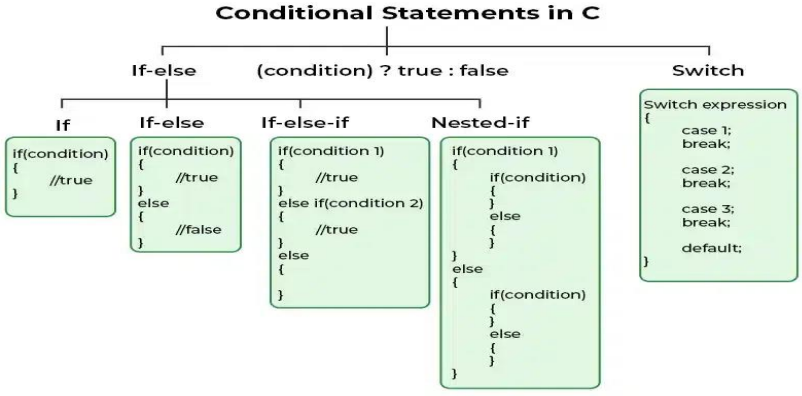
Conditional Statement

- The **conditional statements** (also known as decision control structures) such as if, if else, switch, etc. are used for decision-making purposes in C/C++ programs.
- They are also known as Decision-Making Statements and are used to evaluate one or more conditions and make the decision whether to execute a set of statements or not.
- These decision-making statements in programming languages decide the direction of the flow of program execution.

Need of Conditional Statements

- There come situations in real life when we need to make some decisions and based on these decisions, we decide what should we do next.
- Similar situations arise in programming also where we need to make some decisions and based on these decisions we will execute the next block of code.
- For example, in C if x occurs then execute y else execute z. There can also be multiple conditions like in C if x occurs then execute p, else if condition y occurs execute q, else execute r. This condition of C else-if is one of the many ways of importing multiple conditions.

➔ Types of Conditional Statements in C



1. if

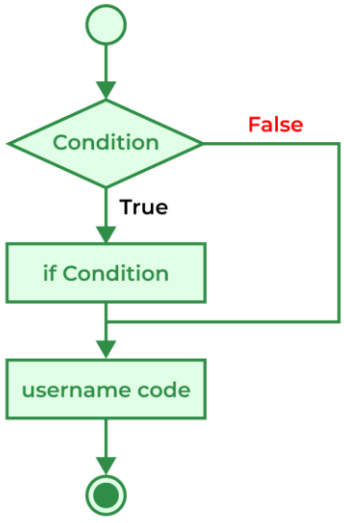
- The if statement is the most simple decision-making statement. It is used to decide whether a certain statement or block of statements will be executed or not i.e if a certain condition is true then a block of statements is executed otherwise not.

Syntax:

```
if
(condition)
{
  // Statements to execute if
  // condition is true
}
```

Here,

- The **condition** after evaluation will be either true or false. C if statement accepts boolean values – if the value is true then it will execute the block of statements below it otherwise not.
- If we do not provide the curly braces '{' and '}' after if(condition) then by default if statement will consider the first immediately below statement to be inside its block.



Example:

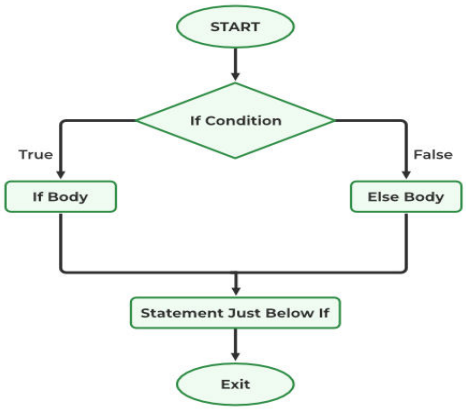
```
#include <stdio.h>

int main()
{

   int gfg = 9;
   // if statement with true condition
   if (gfg < 10) {
      printf("%d is less than 10", gfg);
   }

   // if statement with false condition
   if (gfg > 20) {
      printf("%d is greater than 20", gfg);
   }
return 0;
}
```

Output

```
9 is less than 10
```

2. if-else

- The *if* statement alone tells us that if a condition is true it will execute a block of statements and if the condition is false it won't. But what if we want to do something else when the condition is false?
- Here comes the C *else* statement. We can use the *else* statement with the *if* statement to execute a block of code when the condition is false.



➔Syntax

```
if (condition)
{
  // Executes this block if
   // condition is true
}
else
{
   // Executes this block if
   // condition is false
}
```

Example:

```
#include <stdio.h>
int main()
{
   int i = 20;

   if (i < 15) {
      printf("i is smaller than 15");
   }
   else {
      printf("i is greater than 15");
   }
   return 0;
}
```

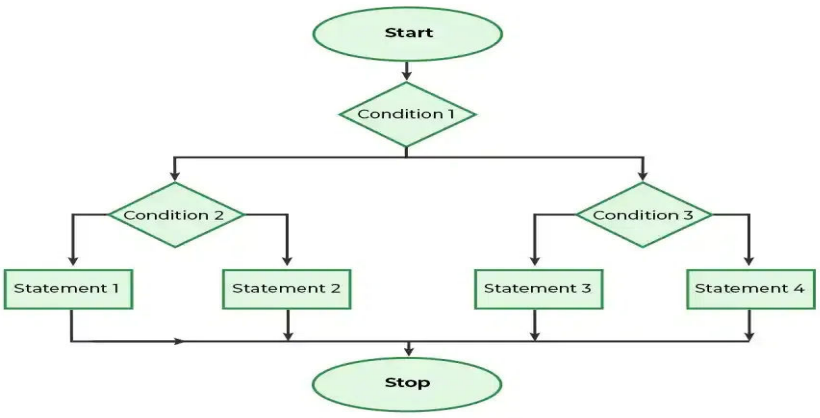
Output: i is greater than 15

3. Nested if-else

- A nested if in C is an if statement that is the target of another if statement.
- Nested if statements mean an if statement inside another if statement. Yes, both C and C++ allow us to nested if statements within if statements, i.e, we can place an if statement inside another if statement.

Syntax

```
if (condition1)
{
  // Executes when condition1 is true
  if (condition2)
  {
    // Executes when condition2 is true
  }
  else
  {
  // Executes when condition2 is false
}
```



Example:

```
// C program to illustrate nested-if statement
#include <stdio.h>
```

```
int main()
{
   int i = 10;

   if (i == 10) {
      // First if statement
      if (i < 15)
         printf("i is smaller than 15\n");

      // Nested - if statement
      // Will only be executed if statement above
      // is true
      if (i < 12)
         printf("i is smaller than 12 too\n");
      else
         printf("i is greater than 15");
   }

   return 0;
}
```

Output:
i is smaller than 15
i is smaller than 12 too

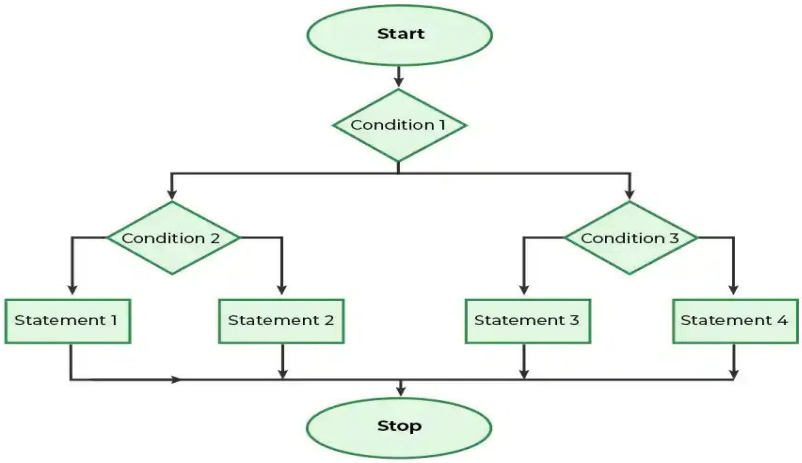
4. if-else-if Ladder

- The if else if statements are used when the user has to decide among multiple options.
- The C if statements are executed from the top down. As soon as one of the conditions controlling the if is true, the statement associated with that if is executed, and the rest of the C else-if ladder is bypassed.

- If none of the conditions is true, then the final else statement will be executed. if-else-if ladder is similar to the switch statement.

Syntax

```
if (condition)
   statement;
else if (condition)
   statement;
.
.

else
   statement;
```



Example

```
// C program to illustrate nested-if statement
#include <stdio.h>
int main()
{
   int i = 20;
```

```
    if (i == 10)
        printf("i is 10");
    else if (i == 15)
        printf("i is 15");
    else if (i == 20)
        printf("i is 20");
    else
        printf("i is not present");
}
```

Output: i is 20

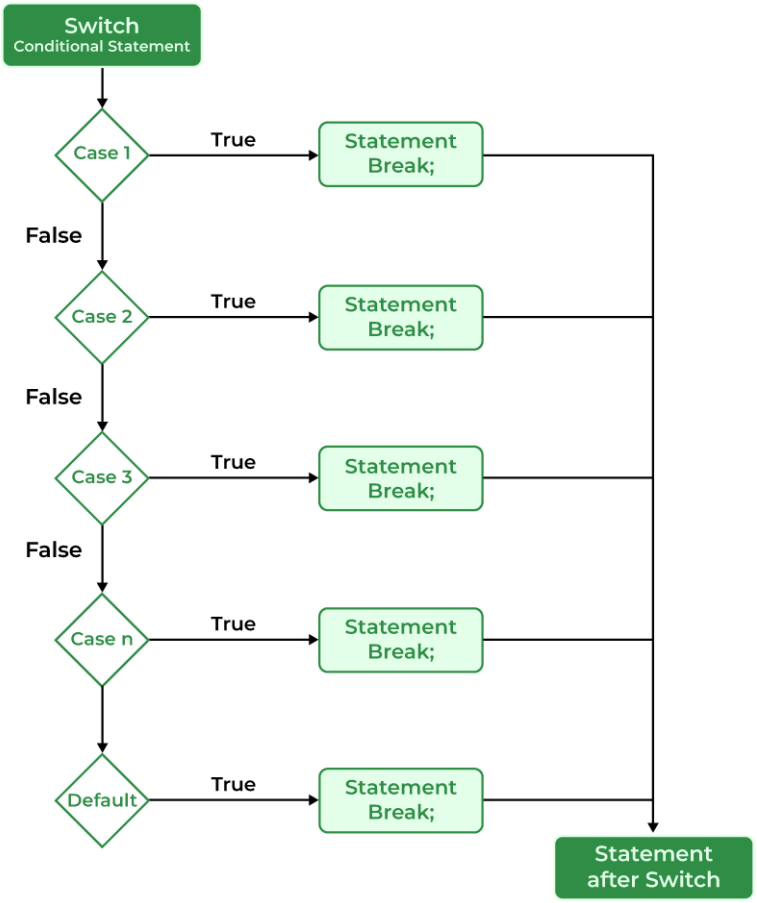
5. Switch Statement

- The switch case statement is an alternative to the if else if ladder that can be used to execute the conditional code based on the value of the variable specified in the switch statement.
- The switch block consists of cases to be executed based on the value of the switch variable.

Syntax

```
switch (expression) {
    case value1:
        statements;
    case value2:
        statements;
    ....
    ....
    ....
    default:
```

Note: *The switch expression should evaluate to either integer or character. It cannot evaluate any other data type.*



Example

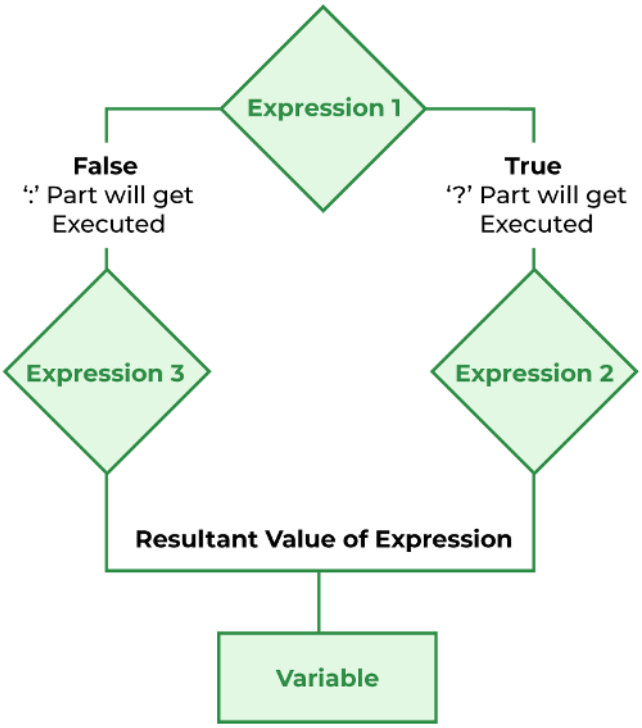
```
// C Program to illustrate the use of switch statement
#include <stdio.h>
int main()
{
   // variable to be used in switch statement
   int var = 2;
   // declaring switch cases
   switch (var) {
   case 1:
      printf("Case 1 is executed");
      break;
   case 2:
      printf("Case 2 is executed");
      break;
```

```
    default:
        printf("Default Case is executed");
        break;
    }
    return 0;
}
```

Output: Case 2 is executed

6. Conditional Operator

- The conditional operator is used to add conditional code in our program. It is like the if-else statement.
- It is also known as the ternary operator as it works on three operands.



Example

```
#include <stdio.h>
// driver code
int main()
{
    int var;
    int flag = 0;
    // using conditional operator to assign the value to var
    // according to the value of flag
    var = flag == 0 ? 25 : -25;
    printf("Value of var when flag is 0: %d\n", var);
    // changing the value of flag
    flag = 1;
   // again assigning the value to var using same statement
    var = flag == 0 ? 25 : -25;
    printf("Value of var when flag is NOT 0: %d", var);

    return 0;
}
```

Output:

```
Value of var when flag is 0: 25
Value of var when flag is NOT 0: -25
```